



Conda for Bioinformatics and Research Computing

Creating reproducible software environments on Mac and Windows

ASI WA-branch workshop 9th July 2026

Dr Jennifer Currenti

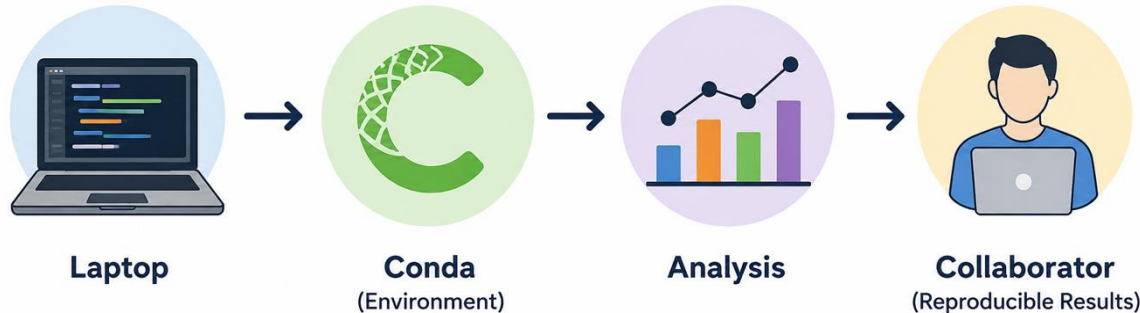


Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

By the end of today you will be able to:

- ✓ Create Conda environments
- ✓ Install software packages
- ✓ Export and share environments
- ✓ Reproduce an analysis on another computer
- ✓ Troubleshoot common problems



The Problem

You install package A



Later you install package B

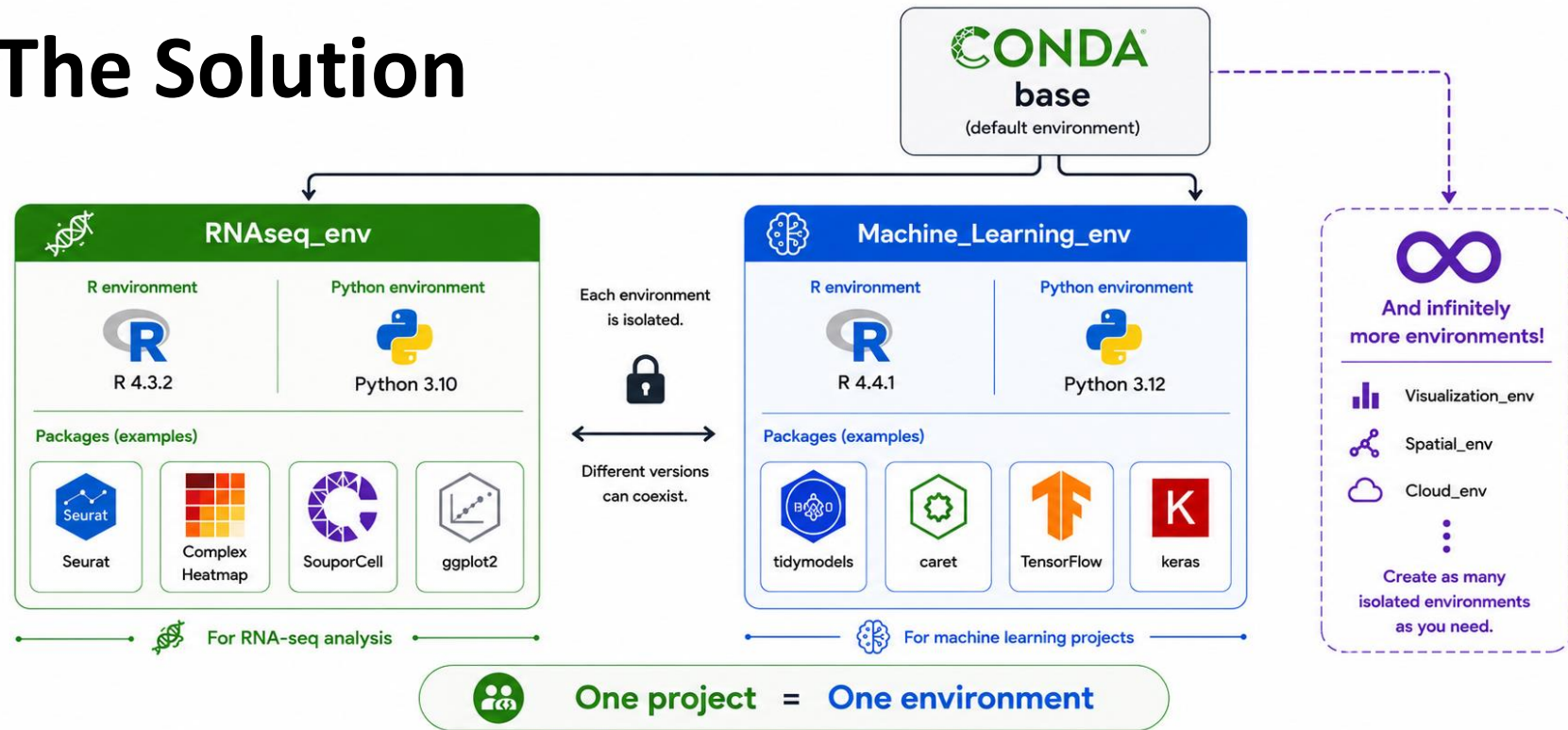


<< Suddenly Package A stops working >>

Why? Dependencies changed



The Solution



Why?

Different projects need different package versions. Environments keep them separate and reproducible.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

When would you use Conda?

When a project depends on specific software packages or software versions.



Projects with software dependencies

Different projects require different package versions



Collaboration

Share the same software environment with colleagues



Reproducibility

Recreate analyses months or years later



HPC and cloud computing

Run the same environment anywhere



Specialised software

Bioinformatics

Machine learning

Statistics

Visualisation



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

ONE-TIME SETUP

Do this once per machine



1 Install Conda (Miniforge)

- Download Miniforge for your operating system
- Run the installer and accept defaults



2 Open Terminal / Prompt

- macOS: open Terminal
- Windows: open Miniforge Prompt from Start Menu



3 Verify Installation

- Check Conda is installed correctly

```
conda --version
```



You're ready!

Move on to environment management.

REPEATABLE WORKFLOW

Create new environments or adjust existing ones



4 Create a New Environment

Create an isolated environment with a name and (optionally) a Python version.

```
conda create -n myenv python=3.10
```



5 Activate the Environment

Activate the environment you want to work in.

```
conda activate myenv # macOS / Linux  
conda activate myenv # Windows
```



9 Remove Environment (optional)

Remove an environment you no longer need.

```
conda env remove -n myenv
```



8 Deactivate Environment

Exit the active environment and return to the base (environment).

```
conda deactivate
```



7 List Environments (optional)

See all environments on your system.

```
conda env list
```



6 Add / Remove / Update Packages

Install new packages or update existing ones.

```
conda install package_name  
conda update package_name  
conda remove package_name
```

Repeat steps 4-9
to create new
environments or
modify existing
ones as needed.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

TASK	COMMAND	WHAT IT DOES	macOS / Linux (Terminal) Example	Windows (Command Prompt / PowerShell) Example
 Show current location	<code>pwd</code>	Print working directory (shows your current path)	<code>\$ pwd</code> /Users/jen/projects	<code>C:\> cd</code> C:\Users\jen\projects
 List files and folders	<code>ls</code>	List items in the current directory	<code>\$ ls</code> data/ notebooks/ README.md	<code>C:\> dir</code> data notebooks README.md
 Change directory	<code>cd <path></code>	Change to the specified directory	<code>\$ cd projects/workshop</code> (now in ~/projects/workshop)	<code>C:\> cd projects\workshop</code> (now in C:\Users\jen\projects\workshop)
 Go up one directory	<code>cd ..</code>	Move up one level in the directory tree	<code>\$ cd ..</code> (moves up one level)	<code>C:\Users\jen\projects> cd ..</code> (moves up one level)
 Go to home directory	<code>cd ~</code>	Go to your home directory	<code>\$ cd ~</code> (now in /Users/jen)	<code>C:\Users\jen\projects> cd %USERPROFILE%</code> (now in C:\Users\jen)
 Create directory	<code>mkdir <name></code>	Create a new directory	<code>\$ mkdir data</code> (creates folder: data)	<code>C:\> mkdir data</code> (creates folder: data)
 Create an empty file	<code>touch <file></code>	Create an empty file	<code>\$ touch notes.txt</code> (creates file: notes.txt)	<code>C:\> type nul > notes.txt</code> (creates file: notes.txt)
 Edit a text file	<code>nano <file></code>	Edit a text file in the terminal	<code>\$ nano notes.txt</code> (opens file: notes.txt)	<code>C:\> notepad notes.txt</code> (opens file: notes.txt in Notepad)
 Remove file (be careful!)	<code>rm <file></code>	Delete (remove) a file	<code>\$ rm notes.txt</code> (deletes notes.txt)	<code>C:\> del notes.txt</code> (deletes notes.txt)
 Remove directory (be careful!)	<code>rm -r <dir></code>	Delete (remove) a directory and its contents	<code>\$ rm -r old_folder</code> (deletes old_folder and everything in it)	<code>C:\> rmdir /s /q old_folder</code> (deletes old_folder and everything in it)
 Get help	<code>--help</code>	Show help for a command	<code>\$ ls --help</code>	<code>C:\> dir /?</code>



Tips

- Paths: macOS/Linux uses forward slashes (/) e.g., `cd projects/data`
- Many commands (like `ls`, `cd`, `mkdir`) work the same in both environments.

- Windows uses backslashes (\\) e.g., `cd projects\data`
- Command line is case sensitive on macOS/Linux; mostly not on Windows (but paths can be).



This is a quick reference. You can always get more information by typing `<command> --help` (macOS/Linux) or `<command> /?` (Windows). | Nano: press `Ctrl + O`, then `Enter` to save; press `Ctrl + X` to exit.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand



Activity

1. What is the current working directory? What files/folders are in this directory? Can you move between folders?
2. Create a new folder for today's workshop and make this the working directory.
3. Did it change correctly? Is there anything in the new folder?



1. Install Conda (Miniforge)

 macOS



1 Download Miniforge

Choose the installer for your Mac:

- Apple Silicon → arm64
- Intel Mac → x86_64



2 Run the installer

Open the downloaded installer and accept the default options.



3 Open Terminal

You will use Terminal for the workshop.

 Windows



1 Download Miniforge

Choose the Windows

x86_64 installer.



2 Run the installer

Open the downloaded installer and accept the default options.



3 Open Miniforge Prompt

Start menu → Miniforge → Miniforge Prompt

 **Already have Anaconda?** You do not need to install Miniforge as well. Use Anaconda Prompt instead; the commands in this workshop are the same.

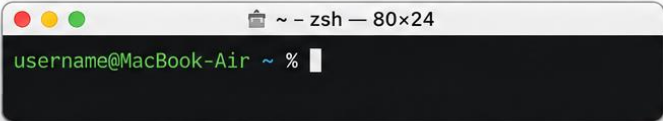
 Download Miniforge: conda-forge.org/miniforge



2. Open Terminal/Prompt

 **macOS**

1. Open Terminal
Applications > Utilities > Terminal



```
username@MacBook-Air ~ %
```

2. Example prompt

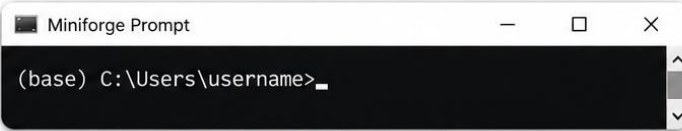
```
username@MacBook-Air ~ %
```

Notes

- Uses zsh shell by default
- Home directory shown as ~
- Case-sensitive file paths

 **Windows**

1. Open Miniforge Prompt
Start Menu > Miniforge > Miniforge Prompt



```
(base) C:\Users\username>
```

2. Example prompt

```
(base) C:\Users\username>
```

Notes

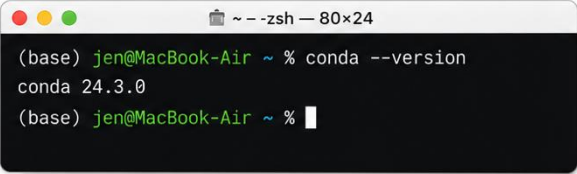
- Uses conda base environment by default
- Paths use backslashes \
- Generally case-insensitive file paths

i Windows users: If you installed Anaconda Distribution instead of Miniforge, use Anaconda Prompt. The Conda commands in this workshop are the same.



3. Verify Installation

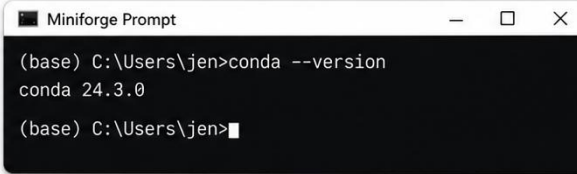
 macOS

- 1. Open Terminal**
Applications > Utilities > Terminal
- 2. Run the version check**
`conda --version`
- 3. You should see a version number**


```
(base) jen@MacBook-Air ~ % conda --version
conda 24.3.0
(base) jen@MacBook-Air ~ %
```

 **Success!**
Conda is installed and ready to use.

 Windows

- 1. Open Miniforge Prompt**
Start Menu > Miniforge > Miniforge Prompt
- 2. Run the version check**
`conda --version`
- 3. You should see a version number**


```
(base) C:\Users\jen>conda --version
conda 24.3.0
(base) C:\Users\jen>
```

 **Success!**
Conda is installed and ready to use.



Tip: If you get a `command not found` error, make sure Miniforge is installed and that you opened the correct terminal.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

4-5. Creating an environment

4 Create a New Environment

Create an isolated environment with a name and (optionally) a Python version.



macOS / Linux

Terminal

```
$ conda create -n  
myenv python=3.10
```



Creates environment
myenv with Python 3.10



Windows

Command Prompt

```
C:\> conda create -n  
myenv python=3.10
```



Creates environment
myenv with Python 3.10

5 Activate the Environment

Activate the environment you want to work in.



macOS / Linux

Terminal

```
$ conda activate  
myenv
```



Your prompt will change
to show the active env.

```
(myenv) $
```



Windows

Command Prompt

```
C:\> conda activate  
myenv
```



Your prompt will change
to show the active env.

```
(myenv) C:\>
```



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Understanding the Prompt



macOS

1. Default prompt (base environment)

```
~ -- zsh -- 80x24  
(base) jen@MacBook-Air ~ %
```

2. After activating a project environment (workshop)

```
~ -- zsh -- 80x24  
(workshop) jen@MacBook-Air ~ %
```

What the important parts mean

Part	Meaning
(base) / (workshop)	The currently active Conda environment
jen@MacBook-Air	Your computer name (and user)
~	Your home folder
%	The terminal is ready for your next command



The key thing to check: the name in brackets at the start changes from (base) to (workshop) when you activate your environment.



Windows

1. Default prompt (base environment)

```
Miniforge Prompt  
(base) C:\Users\jen>
```

2. After activating a project environment (workshop)

```
Miniforge Prompt  
(workshop) C:\Users\jen>
```

What the important parts mean

Part	Meaning
(base) / (workshop)	The currently active Conda environment
C:\Users\jen	Your current folder location
>	The terminal is ready for your next command

Remember: base is Conda's default environment.
Use it to manage Conda itself; create and activate a separate environment for each project.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand



Activity

1. Create an environment called ASI_workshop
2. Activate the environment



You are done when:

- 1 your prompt begins with (ASI_workshop)

```
(ASI_workshop) user@MacBook ~ %
```

- 2 conda env list shows ASI_workshop with a *

```
$ conda env list
# conda environments:
#
base                /Users/user/miniforge3
ASI_workshop        * /Users/user/miniforge3/envs/ASI_workshop
```



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

What is Installed Right Now?

A new environment starts with very little

After creating and activating a new environment, it does **not** automatically contain R, Python, or the packages you use for analysis.



Run: conda list

What you will probably see

- A short list of **Conda's core supporting packages**
- The **Python version**, if you requested one when creating the environment
- Possibly pip, if it was included during creation
- **No R packages or analysis tools yet** unless you installed them



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

6. Managing Packages

6 Add / Remove / Update Packages

Install new packages or update/remove existing ones.

 macOS / Linux

 Terminal

```
$ conda install package_name
$ conda update package_name
$ conda remove package_name
```

 Examples:
package_name = numpy, pandas,
scikit-learn, etc.

 Windows

 Command Prompt

```
C:\> conda install package_name
C:\> conda update package_name
C:\> conda remove package_name
```

 Examples:
package_name = numpy, pandas,
scikit-learn, etc.



Installation

To install this package, run one of the following:

Conda 

```
$ conda install conda-forge::r-seurat
```

Installation

To install this package, run one of the following:

Conda 

```
$ conda install bioconda::r-seurat
```



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

6. Managing Packages

1

```
(ASI_workshop) jennifercurrenti@ ~ % conda install conda-forge::r-heatmap
```

2

```
Channels:
- conda-forge
- bioconda
- defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 25.9.0
  latest version: 26.5.3

Please update conda by running

  $ conda update -n base -c conda-forge conda

## Package Plan ##

  environment location: /Users/jennifercurrenti/miniconda3/envs/ASI_workshop
  added / updated specs:
    - conda-forge::r-heatmap

The following packages will be downloaded:

  package                        | build                | size  | source
  -----|-----|-----|-----
  r-heatmap-1.0.13              | r45hc72bb7e_1       | 92 KB | conda-forge
  -----|-----|-----|-----
  Total:                        |                      | 92 KB |

The following NEW packages will be INSTALLED:

  r-heatmap                conda-forge/noarch::r-heatmap-1.0.13-r45hc72bb7e_1

Proceed ([y]/n)? y
```

3

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done



Errors indicating Conda couldn't complete the installation:

- ✗ PackagesNotFoundError
- ✗ UnsatisfiableError
- ✗ CondaHTTPError
- ✗ EnvironmentNotFound

Key Takeaways

- Always install packages into the correct environment
- Conda may install additional dependencies automatically
- Wait until you see "**Executing transaction: done**" before assuming the installation has finished
- Most of the text is simply progress information, not something you need to interpret



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Where does Conda get software?

Channel	What it is	When you might use it
defaults	Conda's default package channel	General packages
conda-forge	Community-maintained package channel	Many current scientific R/Python packages
Bioconda	Bioinformatics package channel	Tools such as samtools, bcftools and souporcell
Mamba	Faster Conda-compatible installer	When solving/installing is slow

A *channel* is simply a software repository that Conda searches when installing packages.



Installing Packages From Other Sources

Conda environments can use more than Conda channels

- ◇ *Use Conda first for packages available through your chosen channels.*
- ◇ *If a package is not available there, you can install it from another source inside the active Conda environment.*

Recommended order:

1. Create and activate the Conda environment.
2. Install as much as possible with Conda.
3. Add packages from pip, CRAN, Bioconductor, or GitHub only if needed.
4. Record the final environment so it can be recreated later.



Installing Packages From Other Sources

SOURCE	BEST FOR	macOS / Linux (Terminal) Example	Windows (Command Prompt / PowerShell) Example
 Conda channels	R, Python, command-line tools, compiled dependencies	<pre>\$ conda install -c conda-forge pandas</pre>	<pre>C:\> conda install -c conda-forge pandas</pre>
 PyPI with pip	Python packages not available through Conda	<pre>\$ python -m pip install package_name</pre>	<pre>C:\> python -m pip install package_name</pre>
 CRAN	R packages not available through Conda	<pre>\$ R > install.packages("package_name")</pre>	<pre>C:\> R > install.packages("package_name")</pre>
 Bioconductor	Specialised R/Bioconductor packages	<pre>\$ R > BiocManager::install("package_name")</pre>	<pre>C:\> R > BiocManager::install("package_name")</pre>
 GitHub	Development versions or packages not yet released elsewhere	<pre>\$ python -m pip install git+https://github.com/user/repo.git</pre>	<pre>C:\> python -m pip install git+https://github.com/user/repo.git</pre>





Activity

1. Install the following packages into the environment

- a. R
- b. Rstudio
- c. ggPlot2
- d. complexHeatmap - `conda install bioconda::bioconductor-complexheatmap`
- e. pheatmap
- f. patchwork
- g. tidyverse
- h. Here
- i. circlize
- j. viridisLite
- k. gghalves
- l. ggridges

*Useful website for Mac users where a package is not compatible with the M1 chip:
<https://r-universe.dev/search>*

2. What channel did each package use?

3. Run conda list, did all packages install?





Activity - solutions

1. Install the following packages into the environment
 - a. R - `conda install -c r r-package; conda install -c conda-forge r-base; conda install -c conda-forge r-essentials`
 - b. Rstudio - `conda install -c r rstudio`
 - c. ggPlot2 - `conda install conda-forge::r-ggplot2`
 - d. complexHeatmap - `conda install bioconda::bioconductor-complexheatmap`
 - e. theatmap - `conda install conda-forge::r-pheatmap`
 - f. tatchwork - `conda install conda-forge::r-patchwork`
 - g. tidyverse - `conda install conda-forge::r-tidyverse`
 - h. here - `conda install conda-forge::r-here`
 - i. circlize - `conda install conda-forge::r-circlize`
 - j. viridisLite - `conda install conda-forge::r-viridislite`
 - k. gghalves - `conda install conda-forge::r-gghalves`
 - l. ggbridges - `conda install conda-forge::r-ggbridges`



Opening RStudio



macOS

Activate your environment, then open RStudio.

- 1 Activate your environment:

```
conda activate ASI_workshop
```

- 2 Open RStudio:

```
rstudio
```

Check it's working:

RStudio should open and show the correct R version in the Console.

```
R Console
R version 4.4.0 (2024-04-24) -- "Puppy Cup"
...
> sessionInfo()
R version 4.4.0 (2024-04-24)
Platform: aarch64-apple-darwin20
Running under: macOS Sonoma 14.5
...
```



Windows

Activate your environment, then open RStudio.

- 1 Activate your environment:

```
conda activate ASI_workshop
```

- 2 Open RStudio:

```
rstudio
```

Check it's working:

RStudio should open and show the correct R version in the Console.

```
R Console
R version 4.4.0 (2024-04-24) -- "Puppy Cup"
...
> sessionInfo()
R version 4.4.0 (2024-04-24)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 22631)
...
```



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Making an Environment File



1 You write a recipe (from scratch)

Create a simple environment.yml with the packages and versions you want.

How it works



You choose the packages (and optionally versions and channels) and write them in a YAML file.

Example command (not required to create the file):

```
# Create the environment from your recipe
conda env create -f environment.yml
```

Example environment.yml (recipe)

```
name: ASI_workshop
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.11
  - r-base=4.4
  - r-ggplot2
  - bioconductor-complexheatmap
  - pip
  - pip:
    - scanpy
```

VS



2 Conda exports a snapshot (from an existing environment)

Conda looks at everything that is currently installed and records it to a file.

How it works



You create or update an environment, then export the exact packages and versions that are installed.

Example command:

```
conda env export -n ASI_workshop
> environment.yml
```

Example environment.yml (snapshot)

```
name: ASI_workshop
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.11.7=h9e4cc4f_0
  - r-base=4.4.1=h1ddf046_0
  - r-ggplot2=3.5.1=r44h6115d3f_0
  - ... (many more packages)
  - ...
```

	Recipe (you write it)	Snapshot (Conda exports it)
 Goal	Specify the key packages your project needs.	Record everything that is currently installed.
 Best for	Sharing , portability , and building environments on different systems.	Reproducing an exact working environment for yourself, preservation, or publication.
 File is usually	Shorter and easier to read.	Much longer (includes all packages and exact versions/builds).
 What happens when used to create?	Conda solves dependencies and installs the latest compatible versions .	Conda installs the exact versions you had when it was exported (when available for your platform).



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Understanding environment.yml Files

An **environment.yml** file is a small plain-text file that describes the software environment a project needs.

Section	What it tells Conda
name	What to call the environment
channels	Where to look for packages
dependencies	Which packages and versions to install
pip:	Python packages installed from PyPI rather than Conda

```
1  name: rnaseq_env
2
3  channels:
4    - conda-forge
5    - bioconda
6
7  dependencies:
8    - r-base=4.4
9    - python=3.12
10   - scanpy
11   - pip
12   - pip:
13     - some_python_package
14
```

Labels end with a colon

Dashes create a list

Versions are optional

Indentation matters



Important Distinctions in .yaml Files



Project recipe

(you write this)

```
name: rnaseq_project
channels:
  - conda-forge
  - bioconda
dependencies:
  - r-base=4.4           # pinned (important)
  - r-seurat             # no version (flexible)
  - python=3.12         # pinned (important)
```



Flexible



Portable

Use while developing or sharing a project

Develop /
share



Archive exact
working setup



Exact snapshot

(generated by Conda)

```
name: rnaseq_project
channels:
  - conda-forge
  - bioconda
dependencies:
  - r-base=4.4.3=h3f0b6e6_2
  - r-seurat=5.1.0=r44hdfd78af_0
  - python=3.12.8=he870216_0_cpython
```



Precise



Reproducible

Use to archive a working analysis



Use a concise recipe to **build and share**; save an exact snapshot to **preserve what worked**.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Export an Environment

 macOS

1. Open Terminal
Applications > Utilities > Terminal
2. Activate the environment you want to save
`conda activate rnaseq_env`
3. Save the environment as a YAML file
`conda env export -n rnaseq_env -f environment.yml`
4. You should see a confirmation and a new file

```
(base) jen@MacBook-Air ~ % conda activate rnaseq_env
(rnaseq_env) jen@MacBook-Air ~ % conda env export -n
rnaseq_env -f environment.yml
EnvironmentFileNotFound: 'environment.yml'
(rnaseq_env) jen@MacBook-Air ~ % ls environment.yml
environment.yml
(rnaseq_env) jen@MacBook-Air ~ %
```

 **Success!**
Your environment has been saved to environment.yml.

 Windows

1. Open Miniforge Prompt
Start Menu > Miniforge > Miniforge Prompt
2. Activate the environment you want to save
`conda activate rnaseq_env`
3. Save the environment as a YAML file
`conda env export -n rnaseq_env -f environment.yml`
4. You should see a confirmation and a new file

```
(base) C:\Users\jen>conda activate rnaseq_env
(rnaseq_env) C:\Users\jen>conda env export -n rnaseq_env -f
environment.yml
(rnaseq_env) C:\Users\jen>dir environment.yml
environment.yml
(rnaseq_env) C:\Users\jen>
```

 **Success!**
Your environment has been saved to environment.yml.





Activity

1. Save your environment as a .yml file
2. Where did it save? How can you get it to save in a different location?
3. Open the .yml file and look at the contents. Did it include everything? Can you see package versions?



Using a .yaml File

 macOS

- 1. Open Terminal**
Applications > Utilities > Terminal
- 2. Go to the folder where the .yaml file is saved**
`cd ~/Downloads` # or the folder you used
- 3. Create the environment from the .yaml file**
`conda env create -f environment.yaml`
- 4. Activate the new environment**
`conda activate rnaseq_env` # use the name from the .yaml file
- 5. You should see the environment is active**

```
(base) jen@MacBook-Air ~ % cd ~/Downloads
(base) jen@MacBook-Air ~/Downloads % conda env create -f environment.yaml
Channels:
- conda-forge
- bioconda
Collecting package metadata (current_repodata.json): done
Solving environment: done
...
# To activate this environment, use
#
#     $ conda activate rnaseq_env
(base) jen@MacBook-Air ~/Downloads % conda activate rnaseq_env
(rnaseq_env) jen@MacBook-Air ~/Downloads %
```

 **Success!**
The environment has been created and activated.

 Windows

- 1. Open Miniforge Prompt**
Start Menu > Miniforge > Miniforge Prompt
- 2. Go to the folder where the .yaml file is saved**
`cd %USERPROFILE%\Downloads` REM or the folder you used
- 3. Create the environment from the .yaml file**
`conda env create -f environment.yaml`
- 4. Activate the new environment**
`conda activate rnaseq_env` REM use the name from the .yaml file
- 5. You should see the environment is active**

```
(base) C:\Users\jen\Downloads>conda env create -f environment.yaml
Channels:
- conda-forge
- bioconda
Collecting package metadata (current_repodata.json): done
Solving environment: done
...
# To activate this environment, use
#
#     $ conda activate rnaseq_env
(base) C:\Users\jen\Downloads>conda activate rnaseq_env
(rnaseq_env) C:\Users\jen\Downloads>
```

 **Success!**
The environment has been created and activated.





Activity

1. Download the yml file from <https://github.com/theislab/single-cell-best-practices>
2. Create and activate the environment
3. Generate a new yml file and compare the two.
 - a. What is different?
 - b. Why are they different?



7-9. Closing/removing an environment

7 List Environments (optional)

See all environments on your system.

macOS / Linux

Windows

Terminal

Command Prompt

```
$ conda env list
```

```
C:\> conda env list
```

Displays all installed environments.

Displays all installed environments.

8 Deactivate Environment

Exit the active environment and return to the base (environment).

macOS / Linux

Windows

Terminal

Command Prompt

```
$ conda deactivate
```

```
C:\> conda deactivate
```

Returns you to the base environment.
(base) \$

Returns you to the base environment.
(base) C:\>

9 Remove Environment (optional)

Remove an environment you no longer need.

macOS / Linux

Windows

Terminal

Command Prompt

```
$ conda env remove  
-n myenv
```

```
C:\> conda env remove  
-n myenv
```

Removes the environment myenv and all packages.

Removes the environment myenv and all packages.



Why Not Use Base?



Keep base clean and stable

Use it mainly to run and update Conda itself.



Avoid package conflicts

Packages from unrelated projects can require incompatible versions.



Avoid breaking existing work

Updating a package for one project can unexpectedly affect another.



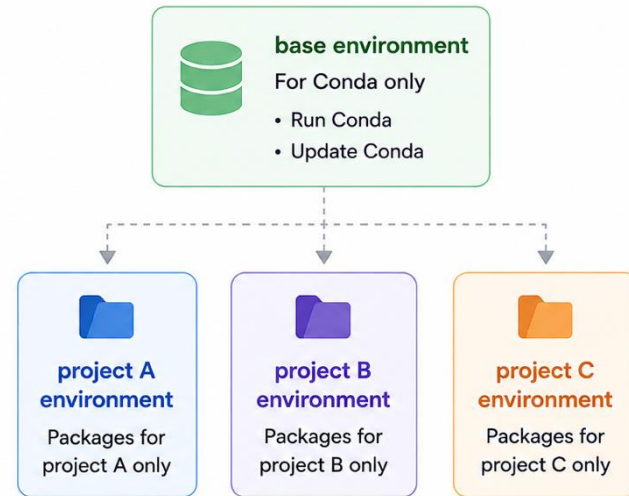
Make projects reproducible

Each project can have its own environment file with the exact software it needs.



Easier to fix problems

If a project environment breaks, delete and recreate it without affecting Conda or your other projects.



Conda best practices

- 1** Use one environment per project



Keep each project's software and dependencies separate.

- 2** Keep base clean



Use base to manage Conda; install project software in dedicated environments.

- 3** Name environments clearly



Use meaningful names such as `pbmc_citeseq`, `bulk_rnaseq`, or `tcr_analysis`.

- 4** Start with a small, intentional package list



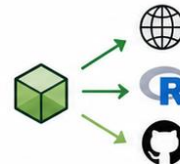
Install the core tools your project needs first; add packages only when required.

- 5** Use Conda channels deliberately



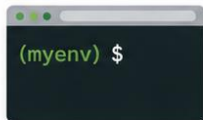
Prefer a consistent channel strategy—e.g., `conda-forge` first, then `bioconda` where needed.

- 6** Install Conda packages first



Use pip, CRAN, Bioconductor, or GitHub only when a package is unavailable through Conda.

- 7** Activate before installing



Check the prompt shows the correct environment before adding packages from any source.

- 8** Record and export environments



Keep an `environment.yml` file with the project so the setup can be recreated or shared.

- 9** Use a concise recipe and an exact snapshot when needed



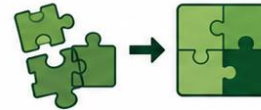
Maintain a portable recipe for day-to-day work; export a fully pinned snapshot for a finished analysis or publication.

- 10** Do not edit an environment casually once it works



Export it first, then make changes deliberately so you can return to the working setup.

- 11** Recreate rather than endlessly repair



If an environment becomes difficult to resolve, rebuild it from the `.yaml` file instead of troubleshooting every conflict individually.



Bottom takeaway: Clean environments, clear names, and saved `.yaml` files make analyses easier to reproduce, share, and revisit.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

	Error / issue	What it usually means	How to fix
	conda: command not found / 'conda' is not recognized	Terminal cannot find Conda	Open Miniforge Prompt on Windows, or restart Terminal after installing Miniforge
(base)	Still seeing (base)	You have not activated your project environment	Run <code>conda activate environment_name</code>
	Activated the wrong environment	Packages are being installed or used in the wrong place	Check with <code>conda env list</code> , then activate the correct one
	Package not found	The package is not in the channels Conda is currently searching	Try <code>conda search package_name</code> , add the correct channel, or check if the Conda package name is different
	Solving environment takes a long time	Conda is trying to find compatible versions	Wait, simplify the install command, install fewer packages at once, or use <code>mamba</code>
	Packages conflict / cannot solve	Requested packages need incompatible versions	Create a fresh environment, pin fewer versions, or install packages in smaller groups
	R/Python package imports fail	Package may not be installed in the active environment	Activate the correct environment, then install the package there
	environment.yml does not work	File may not be in the current folder or has formatting issues	Check <code>pwd / dir</code> , use <code>ls / dir</code> , and check indentation in the .yml file
	Environment already exists	You are trying to create an environment with a name already in use	Use a new name, update the existing environment, or remove the old one
	RStudio uses the wrong R version	RStudio is not pointed at the Conda environment's R	In RStudio, change the R version to the environment's R executable
	Installed into base by mistake	Project packages were added to the default environment	Activate the project environment before installing packages; avoid using <code>base</code> for projects
	Environment feels broken	Too many conflicting changes were made	Export anything important, then recreate the environment from the .yml file



Software Management Tools

Tool	Category	What it does	Typical context
Conda	Package + environment manager	Installs software and creates project-level environments	Day-to-day analysis
pip	Python package manager	Installs Python packages	Usually inside a Conda or <code>venv</code> environment*
Docker	Container platform	Packages an application and its full runtime stack	Local development, deployment, cloud workflows
Apptainer / Singularity	Container platform	Runs portable, self-contained containers without the usual Docker daemon model	Shared HPC systems and scientific workflows

* `venv` environments are isolated directory trees that contains their own Python interpreter, software packages, and scripts

All of these tools help manage software, but they solve different problems



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Conda vs Pip

Feature	Conda	pip
Creates isolated environments	✓ Yes	✗ No*
Installs Python packages	✓ Yes	✓ Yes
Simple Python-only projects	⚠ More setup	✓ Lightweight, standard Python approach
Installs R packages and other software	✓ Yes	✗ No
Manages software dependencies	✓ Strong management	⚠ Limited
Reproducible environments	✓ environment.yml	⚠ requirements.txt (Python only)
Cross-platform package management	✓ Yes	⚠ Python packages only
Common in scientific computing	✓ Very common	⚠ Usually used within Conda envs

*pip is often used inside a virtual environment but does not create Conda-style environments itself.



Recommendation

Use **Conda** to create and manage environments

Use **pip** only when a package is not available through Conda



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Conda vs venv

Feature	Conda	venv
What it is	Package manager and env manager	Lightweight Python env manager
Included with Python	❌ Install separately (e.g. Miniforge)	✅ Included with standard Python
Creates isolated Python environments	✅ Yes	✅ Yes
Installs Python packages	✅ Yes	⚠️ Uses <code>pip</code> alongside <code>venv</code>
Dependency resolution	✅ Across Python, R, binaries, and system libraries	⚠️ Good through modern <code>pip</code> , but Python-only
Supports mixed R/Python workflows	✅ Yes	❌ No
Environment size	⚠️ Often larger	✅ Usually smaller and lighter
Startup / day-to-day simplicity	⚠️ More concepts: channels, solver, environments	✅ Fewer moving parts

● Recommendation

Use **venv** for simple Python-only projects.

Use **Conda** when a project needs more than Python packages—particularly R, command-line tools, or tightly controlled versions across a research workflow.



Conda vs Docker

Feature	Conda	Docker
What it manages	Software packages & envs	Entire application env
Includes operating system	✗ No	✓ Yes
Size	● Lightweight	⚠ Larger
Easy to modify interactively	✓ Yes	⚠ Usually rebuilt
Reproducibility	✓ Good	✓ Excellent
Learning curve	● Lower	⚠ Higher
Complete system isolation	⚠ Shares the host operating system	✓ Includes OS-level libraries and runtime dependencies
Works on HPC systems	✓ Common	⚠ Depends on system

Most researchers will use Conda far more often than Docker.

Docker and Conda are complementary - you can even run Conda environments inside Docker containers.



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand